

MINOR DEGREE PROJECT REPORT

**EMAIL SPAM DETECTION USING
MULTINOMIAL NAIVE BAYES**

A Minor Project Report

submitted in partial fulfillment of the requirements for the award of the

Minor Degree in Machine Learning

Submitted By

POLANA GNANA DEEPIKA

– O210169

GANNABATTULA MANJU SRI MEENAKSHI MEGHANA - O210074



Department of Computer Science Engineering

RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES

ANDHRA PRADESH, INDIA

ONGOLE CAMPUS

Kurnool Road, Ongole, Prakasam District

Andhra Pradesh - 523225

Academic Year

2026–2027

RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES ONGOLE CAMPUS



CERTIFICATE

This is to certify that the project entitled “**Email spam detection using multinomial naive bayes**” being submitted Gannabattula Manju Sri Meenakshi Meghana bearing ID Number O210074 and Polana Gnana Deepika bearing ID Number O210169 in partial fulfilment of the requirements for the award of the Minor Degree in Machine Learning is a Bonafide work carried by them under my supervision and guidance.

Mrs. Sindhu P

Head of Department(i/c)

Department of
Computer Science Engineering

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide for providing valuable guidance, encouragement, and continuous support throughout the completion of this project. I also thank the faculty members of the Department of Electronics and Communication Engineering for their suggestions and motivation.

I am thankful to my institution for providing the necessary facilities and resources required for carrying out this work. I would also like to acknowledge the developers of Python, Google Colab, and the Scikit-learn library for making powerful open-source tools freely available for educational and research purposes.

Finally, I express my heartfelt gratitude to my parents, family members, and friends for their constant encouragement and support during the successful completion of this project.

POLANA GNANA DEEPIKA

– O210169

GANNABATTULA MANJU SRI MEENAKSHI MEGHANA - O210074

EMAIL SPAM DETECTION USING MULTINOMIAL NAIVE BAYES

ABSTRACT

Electronic communication has become one of the most important methods of exchanging information in today's digital world. Millions of emails and SMS messages are sent every day for personal, educational, and business purposes. Along with legitimate communication, a large number of unwanted messages, commonly known as spam, are also transmitted. Spam messages usually contain advertisements, fake offers, phishing links, fraudulent schemes, malware, or misleading information. These unwanted messages not only consume valuable time but also pose serious security and privacy risks to users. Therefore, there is a growing need for an intelligent system capable of automatically identifying and filtering spam messages before they reach the user.

Machine Learning provides an efficient solution to this problem by learning patterns from previously labelled messages. Instead of manually checking every message, a machine learning model can automatically classify incoming messages as spam or legitimate (ham). This project implements an Email Spam Detection System using the Multinomial Naive Bayes algorithm, which is widely recognized for its excellent performance in text classification problems. The algorithm uses the probability of words appearing in spam and non-spam messages to make accurate predictions.

The SMS Spam Collection Dataset is used for training and testing the model. The dataset contains 5,572 labelled SMS messages categorized into two classes: spam and ham. Before training the model, the dataset undergoes preprocessing steps such as removing unnecessary columns, renaming attributes, converting class labels into numerical values, and transforming textual data into numerical feature vectors using the Term Frequency-Inverse Document Frequency (TF-IDF) technique. The processed data is then divided into training and testing datasets to evaluate the model's performance.

The proposed system achieves an accuracy of approximately 96.28%, demonstrating that the Multinomial Naive Bayes classifier is highly effective for spam message detection. The developed model successfully identifies unwanted messages while minimizing the misclassification of legitimate messages. The project highlights the practical application of machine learning in cybersecurity and communication systems and demonstrates how intelligent text classification techniques can improve digital communication by reducing spam.

Table of Contents

Chapter 1

Introduction

1.1.Introduction

1.2.Need for the Project

1.3.Problem Statement

1.4.Objectives

1.5.Scope of the Project

Chapter 2

Literature Survey

2.1.Introduction

2.2.Traditional Spam Detection Methods

2.3.Machine Learning in Spam Detection

2.4.Review of Existing Machine Learning Algorithms

2.5.Why Multinomial Naive Bayes was Selected

2.6.Research Gap

2.7Summary

Chapter 3

System Analysis

3.1.Introduction

3.2.Existing System

3.3.Limitations of Existing System

3.4.Proposed System

3.5.Advantages of Proposed System

3.6.Feasibility Study

3.7.System Requirements

3.8.System Architecture

3.9.Summary

Chapter 4

System Design and Methodology

4.1.Introduction

4.2.Methodology

4.3.Data Collection

4.4.Data Preprocessing

4.5.Feature Extraction

4.6.Train-Test Split

4.7.Model Training

4.8.Model Testing

4.9.Performance Evaluation

4.10.Prediction Process

4.11.Summary

Chapter 5

Implementation

5.1.Introduction

5.2.Development Environment

5.3.Python Libraries Used

5.4.Implementation Steps

5.5.Output Screens

5.6.Results

5.7.Summary

Chapter 6

Results and Discussion

6.1.Introduction

6.2.Experimental Results

6.3.Performance Evaluation

6.4.Confusion Matrix Analysis

6.5.Prediction Results

6.6.Discussion

6.7.Summary

Chapter 7

Conclusion and Future Scope

7.1.Conclusion

7.2.Future Scope

References

CHAPTER 1

INTRODUCTION

1.1 Introduction

The rapid advancement of internet technologies and digital communication has significantly transformed the way people interact and exchange information. Email and Short Message Service (SMS) have become essential communication tools for individuals, educational institutions, businesses, banks, and government organizations. Every day, billions of messages are transmitted worldwide for sharing personal information, business updates, promotional content, and important notifications. However, this tremendous growth in digital communication has also led to a substantial increase in spam messages.

Spam messages are unsolicited and unwanted messages sent to a large number of recipients without their consent. These messages often include advertisements, fake lottery notifications, fraudulent financial offers, phishing links, malware, and various types of cyber scams. Spam messages not only waste users' valuable time but also consume network resources and create serious security threats. Many cybercriminals use spam emails and SMS messages to steal confidential information such as passwords, banking details, and personal identity information.

Traditionally, spam filtering systems relied on manually defined rules and keyword matching techniques. Although these methods were useful in the early stages, they became less effective as spam messages evolved. Modern spam messages frequently use new vocabulary, modified sentence structures, and hidden malicious links that can easily bypass traditional filtering systems. Therefore, more intelligent techniques are required to detect spam accurately.

Machine Learning has emerged as one of the most effective technologies for solving classification problems such as spam detection. Machine learning algorithms can automatically learn patterns from previously labelled data and make predictions on new unseen messages without being explicitly programmed for every possible situation. This makes machine learning highly suitable for detecting spam messages because it continuously improves its prediction capability based on available training data.

Among the various machine learning algorithms, the Multinomial Naive Bayes classifier is considered one of the most efficient algorithms for text classification problems. It is computationally fast, easy to implement, and produces excellent classification accuracy even with large datasets. Since spam detection primarily involves analyzing textual information, the Multinomial Naive Bayes algorithm becomes an ideal choice for this project.

In this project, an Email Spam Detection System is developed using Python programming language and the Google Colab environment. The SMS Spam Collection Dataset is used for training and testing the model. Text messages are converted into numerical representations using the TF-IDF Vectorizer, after which the Multinomial Naive Bayes classifier is trained.

Finally, the model predicts whether a new message belongs to the spam category or the legitimate (ham) category.

The developed system provides a simple, efficient, and reliable approach for spam detection. It demonstrates how machine learning techniques can significantly improve communication security while reducing the burden of manually identifying unwanted messages.

1.2 Need for the Project

With the increasing use of smartphones and internet-based communication services, spam messages have become one of the major challenges in digital communication. Every day, users receive numerous promotional messages, fake advertisements, phishing emails, lottery scams, and fraudulent financial offers. These unwanted messages create inconvenience and increase the possibility of cyberattacks.

Manual identification of spam messages is time-consuming and inefficient, especially when dealing with thousands of messages every day. Traditional rule-based spam filters often fail to detect newly emerging spam patterns because they depend on predefined keywords and manually designed rules.

Machine learning provides an intelligent alternative by learning patterns directly from historical data. It can automatically classify messages based on their content and continuously improve its prediction capability. Therefore, developing an Email Spam Detection System using Machine Learning is both practically useful and academically valuable.

1.3 Problem Statement

The objective of this project is to develop a machine learning model capable of automatically classifying SMS or email messages into two categories: Spam and Ham (Not Spam). The proposed system uses the Multinomial Naive Bayes algorithm to learn patterns from labelled messages and accurately predict whether an incoming message is legitimate or spam. The system aims to reduce unwanted communication, improve user security, and increase the efficiency of digital communication systems.

1.4 Objectives

The major objectives of this project are:

To study the concept of spam message classification using machine learning.

To preprocess the SMS Spam Collection Dataset for model training.

To convert textual data into numerical features using TF-IDF Vectorization.

To implement the Multinomial Naive Bayes classification algorithm.

To evaluate the performance of the model using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.

To develop a prediction system capable of classifying new messages as Spam or Ham.

To understand the practical application of machine learning in text classification and cybersecurity.

1.5 Scope of the Project

The scope of this project is limited to detecting spam messages using machine learning techniques. The system is trained on the SMS Spam Collection Dataset and predicts whether an incoming message belongs to the spam category or the legitimate category. The project demonstrates the practical implementation of text classification algorithms and can be extended to real-world email filtering systems, mobile messaging applications, banking alerts, customer support systems, and cybersecurity applications. Future improvements may include multilingual spam detection, deep learning techniques, and real-time deployment in communication platforms.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

The rapid growth of internet-based communication has significantly increased the volume of email and SMS traffic worldwide. Along with genuine communication, the number of spam messages has also increased considerably. Spam messages usually contain advertisements, phishing links, fake promotional offers, malware, and fraudulent information intended to mislead users. Due to these increasing threats, researchers have proposed various techniques to automatically identify and filter spam messages.

Initially, spam detection systems were based on manually designed rules and keyword matching methods. These approaches depended on predefined lists of suspicious words or phrases. Although they were simple to implement, they were unable to detect newly emerging spam patterns because spammers continuously modify message content to avoid detection. As a result, traditional filtering techniques gradually became ineffective.

Machine Learning has emerged as one of the most promising solutions for spam detection. Unlike traditional rule-based systems, machine learning algorithms learn directly from historical data and automatically identify hidden patterns within messages. These algorithms can generalize well to new unseen messages, making them highly suitable for real-world spam detection applications.

Researchers have successfully applied several machine learning algorithms to classify spam messages. These include Naive Bayes, Logistic Regression, Decision Trees, Support Vector Machines (SVM), Random Forest, K-Nearest Neighbors (KNN), and Artificial Neural Networks. Among these methods, the Multinomial Naive Bayes classifier has consistently demonstrated excellent performance in text classification tasks because of its simplicity, speed, and high accuracy.

The present project focuses on implementing the Multinomial Naive Bayes algorithm for spam detection using the SMS Spam Collection Dataset. The project demonstrates how text preprocessing and feature extraction techniques improve the accuracy of machine learning models.

2.2 Traditional Spam Detection Methods

Before the introduction of machine learning, spam filtering was primarily performed using rule-based techniques. In these methods, system administrators manually defined rules based on specific keywords, sender addresses, blocked domains, or suspicious message patterns.

For example, messages containing words such as "Congratulations", "Lottery", "Free", "Winner", or "Prize" were automatically classified as spam. Similarly, emails received from blacklisted domains were blocked without analyzing their content.

Although rule-based systems were simple to implement, they suffered from several limitations. Spammers frequently changed spellings, inserted special characters, or modified message structures to bypass keyword filters. As the number of spam messages increased, maintaining these manually created rules became difficult and time-consuming. Consequently, researchers began exploring intelligent data-driven approaches for spam detection.

2.3 Machine Learning in Spam Detection

Machine Learning has revolutionized spam filtering by enabling computers to learn directly from data instead of relying on manually written rules. A machine learning model is trained using previously labelled examples of spam and legitimate messages. During training, the model learns statistical relationships between words and their corresponding message categories.

After training, the model can predict whether a new incoming message is spam or ham. Since machine learning algorithms continuously learn from data, they can successfully identify new spam patterns that traditional rule-based systems often fail to detect.

The success of machine learning in spam detection depends largely on three major stages:

Data preprocessing

Feature extraction

Classification algorithm

Proper preprocessing removes unnecessary information from the dataset and converts raw text into a suitable format. Feature extraction techniques such as TF-IDF transform textual messages into numerical vectors that machine learning algorithms can process. Finally, classification algorithms learn patterns from these vectors and perform accurate predictions.

2.4 Review of Existing Machine Learning Algorithms

Several machine learning algorithms have been widely used for spam message classification. Some of the most popular algorithms are discussed below.

Naive Bayes

Naive Bayes is one of the oldest and most successful probabilistic classification algorithms. It calculates the probability that a message belongs to either the spam or ham category based on the occurrence of individual words. The algorithm assumes that all words contribute independently to the prediction. Although this assumption is not always true, Naive Bayes performs remarkably well for text classification problems because of its computational efficiency and simplicity.

Logistic Regression

Logistic Regression is a supervised machine learning algorithm commonly used for binary classification problems. It estimates the probability that a message belongs to a particular

class using a logistic function. Logistic Regression provides good accuracy but generally requires more computational resources than Naive Bayes.

Support Vector Machine (SVM)

Support Vector Machine is another popular supervised learning algorithm used for text classification. It identifies the optimal boundary that separates spam and legitimate messages. SVM often achieves high accuracy but requires longer training time and more computational power, particularly for large datasets.

Decision Tree

Decision Tree algorithms classify messages by repeatedly splitting the dataset into smaller subsets based on selected features. Although decision trees are easy to interpret, they may suffer from overfitting if the tree becomes excessively complex.

Random Forest

Random Forest combines multiple decision trees to improve classification accuracy and reduce overfitting. It generally performs better than individual decision trees but requires greater computational resources.

2.5 Why Multinomial Naive Bayes was Selected

Among all the available machine learning algorithms, Multinomial Naive Bayes was selected for this project because it is particularly designed for text classification problems. The algorithm efficiently handles word frequency information extracted using TF-IDF Vectorization.

Some important advantages of Multinomial Naive Bayes include:

High accuracy for text classification tasks.

Fast training and prediction speed.

Simple implementation using Python libraries.

Low memory requirements.

Suitable for both small and large datasets.

Excellent performance with sparse text data.

These advantages make Multinomial Naive Bayes one of the most widely used algorithms in spam filtering systems.

2.6 Research Gap

Although many spam detection techniques have been proposed, several challenges still remain. Spam messages continuously evolve as attackers develop new strategies to bypass filtering systems. Many existing systems are designed for specific datasets and may not perform equally well on different communication platforms.

Some machine learning models achieve high accuracy but require extensive computational resources and complex parameter tuning. Therefore, there is a need for lightweight, efficient, and easily deployable spam detection systems that maintain high accuracy while minimizing computational complexity.

The proposed system addresses this requirement by implementing the Multinomial Naive Bayes algorithm, which provides an excellent balance between computational efficiency and prediction accuracy. The achieved accuracy of approximately 96.28% demonstrates that the proposed approach effectively classifies spam and legitimate messages while remaining computationally simple.

2.7 Summary

This chapter presented a comprehensive review of existing spam detection techniques and machine learning approaches used in text classification. Traditional rule-based filtering methods suffer from poor adaptability and limited detection capability. Machine learning techniques overcome these limitations by learning directly from labelled data. Several classification algorithms have been studied, including Logistic Regression, Support Vector Machine, Decision Tree, Random Forest, and Naive Bayes. After comparing their characteristics, the Multinomial Naive Bayes algorithm was selected for the proposed system because of its simplicity, fast execution, and excellent performance in text classification tasks. The next chapter discusses the existing system, proposed system, and overall system analysis in detail.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Introduction

System analysis is one of the most important phases in software development. It involves understanding the existing system, identifying its limitations, and proposing a better solution to overcome those problems. In this project, the system analysis focuses on understanding the challenges associated with spam message detection and designing an efficient machine learning-based solution.

Spam messages have become a major concern in digital communication. They consume users' time, increase storage requirements, and may contain malicious links that lead to phishing attacks, financial fraud, or malware installation. Therefore, an intelligent system capable of automatically detecting spam messages is essential.

The proposed Email Spam Detection System uses the Multinomial Naive Bayes algorithm to classify SMS or email messages as either Spam or Ham (Not Spam). The system analyzes message content, learns patterns from historical data, and predicts the category of new incoming messages with high accuracy.

3.2 Existing System

Before the development of machine learning-based spam detection systems, spam filtering mainly relied on manual rule-based techniques. These systems used predefined rules, blocked keywords, blacklisted email addresses, and manually maintained spam dictionaries to classify messages.

For example, if a message contained words such as "Congratulations," "Winner," "Free Prize," or "Lottery," the system automatically marked it as spam. Similarly, emails received from blacklisted domains were rejected without analyzing their content.

Although these methods were effective for simple spam messages, they failed to detect modern spam techniques. Spammers frequently changed spellings, inserted symbols between words, used new vocabulary, or shortened URLs to bypass these filters. As a result, many spam messages successfully reached users' inboxes.

Moreover, maintaining a large set of manually created filtering rules required continuous updates by system administrators. This increased maintenance costs and reduced overall system efficiency.

3.3 Limitations of Existing System

The existing rule-based spam detection system has several disadvantages:

Depends completely on manually created rules.

Unable to detect newly emerging spam patterns.

Requires continuous maintenance and rule updates.

Produces higher false positive and false negative rates.

Cannot learn automatically from new data.

Difficult to scale for large communication networks.

Less effective against phishing attacks and sophisticated spam messages.

These limitations motivated researchers to adopt machine learning techniques for spam classification.

3.4 Proposed System

The proposed system implements an intelligent spam detection model using the Multinomial Naive Bayes machine learning algorithm. Instead of relying on manually written rules, the system learns directly from a labeled dataset containing spam and ham messages.

Initially, the SMS Spam Collection Dataset is loaded into the system. The dataset undergoes preprocessing, where unnecessary columns are removed, column names are renamed, and class labels are converted into numerical values. The text messages are then transformed into numerical feature vectors using the Term Frequency-Inverse Document Frequency (TF-IDF) technique.

The processed data is divided into training and testing datasets. The training dataset is used to train the Multinomial Naive Bayes classifier, while the testing dataset is used to evaluate the model's performance. Once training is complete, the system predicts whether new incoming messages are spam or ham.

The proposed system achieved an accuracy of 96.28%, indicating excellent performance in text classification. It automatically identifies spam messages while minimizing incorrect classification of genuine messages.

3.5 Advantages of Proposed System

The proposed machine learning-based spam detection system offers several advantages over traditional methods.

High classification accuracy.

Automatic learning from training data.

Fast prediction speed.

Easy implementation using Python.

Low computational complexity.

Minimal memory usage.

Scalable for large datasets.

Suitable for real-time spam detection.

Reduces manual effort.

Improves communication security.

The use of the TF-IDF Vectorizer further enhances the performance by converting text messages into meaningful numerical representations before classification.

3.6 Feasibility Study

Before implementing the project, a feasibility study was conducted to determine whether the proposed system could be successfully developed.

Technical Feasibility

The project is technically feasible because it uses open-source software such as Python, Google Colab, Pandas, and Scikit-learn. These tools provide all the necessary libraries for machine learning implementation without requiring expensive hardware or software.

Economic Feasibility

The project is economically feasible since all required software tools are freely available. Google Colab provides a free cloud-based environment for executing Python code, eliminating the need for costly development infrastructure.

Operational Feasibility

The proposed system is simple to operate. Users only need to enter a message, and the system automatically predicts whether it is spam or ham. No specialized technical knowledge is required to use the application.

Schedule Feasibility

The project can be completed within the allocated academic schedule because the selected algorithm is simple to implement and the dataset is publicly available.

3.7 System Requirements

Hardware Requirements

Processor: Intel Core i3 or above

RAM: Minimum 4 GB

Storage: 20 GB Free Space

Internet Connection

Laptop/Desktop Computer

Software Requirements

Operating System: Windows 10/11

Programming Language: Python

Development Environment: Google Colab

Libraries Used:

Pandas

NumPy

Scikit-learn

Matplotlib

NLTK (Optional)

3.8 System Architecture

The overall working process of the proposed system consists of several sequential stages.

SMS Spam Dataset



Data Preprocessing

(Remove unwanted columns,

Rename columns,

Convert labels)



TF-IDF Feature Extraction



Train-Test Split



Multinomial Naive Bayes Model



Prediction

(Spam / Ham)



Performance Evaluation

Accuracy, Precision,

Recall, F1-Score,

Confusion Matrix

3.9 Summary

This chapter presented a detailed analysis of the proposed spam detection system. The limitations of traditional rule-based filtering methods were discussed, highlighting the need for intelligent machine learning solutions. The proposed system uses the Multinomial Naive Bayes algorithm along with TF-IDF feature extraction to automatically classify spam messages with high accuracy. A feasibility study confirmed that the project is technically, economically, operationally, and schedule-wise feasible. The overall system architecture was also presented to illustrate the complete workflow of the proposed model.

CHAPTER 4

SYSTEM DESIGN AND METHODOLOGY

4.1 Introduction

System design is the process of defining the architecture, components, modules, interfaces, and workflow of a software system. It acts as a bridge between system analysis and implementation by describing how the proposed solution will be developed. In this project, the system is designed to classify SMS or email messages into two categories: Spam and Ham (Not Spam).

The proposed Email Spam Detection System uses Machine Learning to automatically analyze text messages and predict whether they are legitimate or spam. The system follows a sequence of steps that include data collection, preprocessing, feature extraction, model training, testing, prediction, and performance evaluation. Each step contributes to improving the overall accuracy and reliability of the spam detection model.

The design of the system is simple, efficient, and easy to understand. It is implemented using Python in Google Colab, making it suitable for academic learning and experimentation.

4.2 Methodology

The proposed methodology consists of several sequential stages. Each stage performs a specific task that contributes to the final prediction of whether a message is spam or not.

The methodology adopted in this project is illustrated below:

Dataset Collection



Data Preprocessing



Feature Extraction (TF-IDF)



Train-Test Split



Model Training

(Multinomial Naive Bayes)



Model Testing



Prediction

(Spam / Ham)



Performance Evaluation

The above methodology ensures that the machine learning model learns meaningful patterns from historical data and accurately predicts new messages.

4.3 Data Collection

The first step in the proposed system is collecting an appropriate dataset for training and testing the machine learning model. The SMS Spam Collection Dataset is used in this project. It is a publicly available dataset widely used for spam classification research.

The dataset contains 5,572 SMS messages, where each message is labelled as either Spam or Ham (Not Spam). Every record contains two important attributes:

Label – Indicates whether the message is Spam or Ham.

Message – Contains the text content of the SMS.

This dataset provides sufficient examples of both spam and legitimate messages, allowing the model to learn distinguishing patterns effectively.

4.4 Data Preprocessing

Raw datasets often contain unnecessary columns, inconsistent formatting, and missing values. Data preprocessing is therefore performed before training the machine learning model.

The following preprocessing steps were carried out:

Step 1: Loading the Dataset

The dataset was imported into Google Colab using the Pandas library.

Step 2: Removing Unnecessary Columns

The original dataset contained additional unnamed columns that were empty. These columns were removed because they did not contribute to spam detection.

Step 3: Renaming Columns

The column names v1 and v2 were renamed as label and message to improve readability.

Step 4: Label Encoding

The text labels were converted into numerical values.

Ham → 0

Spam → 1

This conversion enables the machine learning algorithm to process the target variable efficiently.

4.5 Feature Extraction

Machine learning algorithms cannot directly understand textual data. Therefore, text messages must first be converted into numerical representations.

This project uses Term Frequency-Inverse Document Frequency (TF-IDF) Vectorization for feature extraction.

TF-IDF measures the importance of each word in a message by considering:

How frequently the word appears in the message.

How rare the word is across the entire dataset.

Words that appear frequently in spam messages but rarely in legitimate messages receive higher importance scores. This enables the classifier to distinguish spam from non-spam messages more effectively.

The TF-IDF Vectorizer converts every SMS message into a numerical feature vector that can be processed by the Multinomial Naive Bayes classifier.

4.6 Train-Test Split

After preprocessing and feature extraction, the dataset is divided into two parts.

Training Dataset (80%)

Testing Dataset (20%)

The training dataset is used to teach the machine learning model. During this stage, the model learns the relationship between the message content and its corresponding class label.

The testing dataset is completely separate from the training data. It is used only after training is complete to evaluate the model's performance on unseen messages.

This separation ensures that the model does not simply memorize the training data but instead learns to generalize effectively.

4.7 Model Training

The Multinomial Naive Bayes algorithm is selected as the primary classification algorithm for this project. It is particularly suitable for text classification because it calculates the probability that a message belongs to a specific class based on the occurrence of words.

The model is trained using the training dataset after the TF-IDF transformation. During training, the classifier learns the statistical relationship between different words and their likelihood of appearing in spam or ham messages.

The training process is computationally efficient and requires very little execution time, making the algorithm suitable for real-time applications.

4.8 Model Testing

After training, the model is evaluated using the testing dataset.

Each testing message is converted into TF-IDF feature vectors using the same transformation applied during training. The trained classifier predicts whether each message belongs to the spam or ham category.

The predicted labels are then compared with the actual labels to determine the effectiveness of the model.

4.9 Performance Evaluation

The performance of the proposed system is measured using several evaluation metrics.

Accuracy

Accuracy measures the percentage of correctly classified messages.

Precision

Precision measures how many messages predicted as spam were actually spam.

Recall

Recall measures how many actual spam messages were correctly identified by the model.

F1-Score

The F1-Score provides a balanced measure by combining both precision and recall.

Confusion Matrix

The confusion matrix provides a graphical representation of correctly and incorrectly classified messages.

Using these evaluation metrics provides a comprehensive understanding of the model's performance.

4.10 Prediction Process

Once the model is successfully trained, it is ready to classify new messages.

The prediction process consists of the following steps:

User enters a message.

The message is converted into TF-IDF features.

The trained Naive Bayes model analyzes the features.

The model calculates the probability of Spam and Ham.

The message is classified as either:

Spam

Not Spam (Ham)

This process is completed within a fraction of a second, demonstrating the efficiency of the machine learning model.

4.11 Summary

This chapter explained the complete design and methodology of the proposed Email Spam Detection System. The system begins with dataset collection, followed by preprocessing, feature extraction using TF-IDF, training and testing of the Multinomial Naive Bayes model, and finally performance evaluation. The modular design of the system makes it simple, efficient, and suitable for text classification tasks. The next chapter presents the implementation details, including the software tools used, Python libraries, code structure, and execution process.

CHAPTER 5

IMPLEMENTATION

5.1 Introduction

Implementation is the stage where the proposed system is developed using programming tools and machine learning libraries. In this project, the Email Spam Detection System was implemented using Python in the Google Colab environment. The implementation includes loading the dataset, preprocessing the data, training the machine learning model, testing its performance, and predicting whether a message is spam or ham.

5.2 Development Environment

The project was developed using the following software and hardware.

Hardware Requirements

Processor: Intel Core i3 or above

RAM: 4 GB or higher

Storage: 20 GB free space

Software Requirements

Operating System: Windows 10/11

Programming Language: Python

Platform: Google Colab

Libraries:

Pandas

NumPy

Scikit-learn

Matplotlib

5.3 Python Libraries Used

The following libraries were used during implementation:

Pandas: Used for loading and manipulating the dataset.

NumPy: Used for numerical operations.

Scikit-learn: Used for TF-IDF Vectorizer, train-test split, Multinomial Naive Bayes model, and evaluation metrics.

Matplotlib: Used to visualize the confusion matrix.

These libraries simplified the implementation and improved development efficiency.

5.4 Implementation Steps

The implementation of the system was carried out in the following steps:

Step 1: Dataset Loading

The SMS Spam Collection dataset (spam.csv) was uploaded into Google Colab and loaded using the Pandas library.

Step 2: Data Preprocessing

The unnecessary columns were removed, and the remaining columns were renamed to label and message. The class labels were converted into numerical values, where Ham = 0 and Spam = 1.

Step 3: Feature Extraction

The message column was converted into numerical vectors using the TF-IDF Vectorizer. This allowed the machine learning model to process text data efficiently.

Step 4: Train-Test Split

The dataset was divided into training and testing sets using an 80:20 ratio.

Step 5: Model Training

The Multinomial Naive Bayes classifier was trained using the training dataset.

Step 6: Prediction

The trained model was tested on the testing dataset and was also used to classify new user-entered messages as Spam or Ham.

5.5 Output Screens

1. Dataset loaded

Email_spam_detection ☆ ☁

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

Files

- ..
- sample_data
- spam.csv

```
[1] ✓ Os
import pandas as pd

# Load the dataset
df = pd.read_csv("spam.csv", encoding='latin-1')

# Display first 5 rows
df.head()
```

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
0	ham	Go until jurong point, crazy.. Available only ...	NaN	NaN	NaN
1	ham	OK lar... Joking wif u oni...	NaN	NaN	NaN
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	NaN	NaN	NaN
3	ham	U dun say so early hor... U c already then say...	NaN	NaN	NaN
4	ham	Nah I don't think he goes to usf, he lives aro...	NaN	NaN	NaN

Next steps: [Generate code with df](#) [New interactive sheet](#)

2. Removed unwanted columns and Rename columns

Email_spam_detection ☆ ☁

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

Files

- ..
- sample_data
- spam.csv

```
[3] ✓ Os
# Remove unnecessary columns
df = df[['v1', 'v2']]

# Rename columns
df.columns = ['label', 'message']

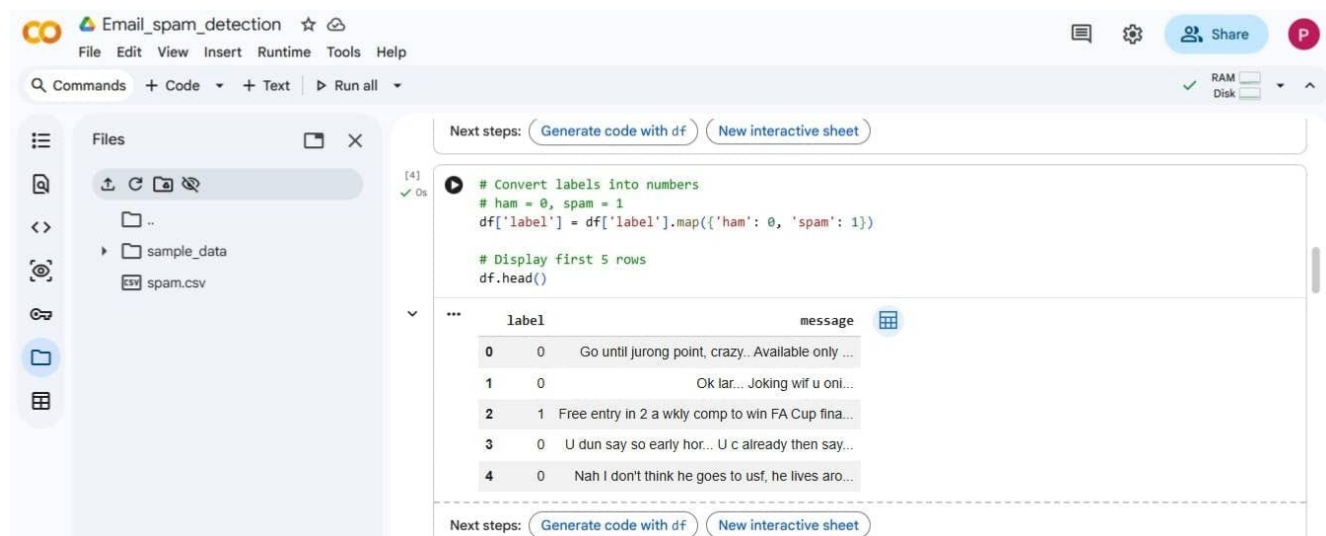
# Display first 5 rows
df.head()
```

	label	message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	OK lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Next steps: [Generate code with df](#) [New interactive sheet](#)

Disk 87.75 GB available

3. Converting labels into numbers



The screenshot shows a Jupyter Notebook titled "Email_spam_detection". The left sidebar displays a file explorer with a folder named "sample_data" containing a file "spam.csv". The main area shows a code cell with the following Python code:

```
# Convert labels into numbers
# ham = 0, spam = 1
df['label'] = df['label'].map({'ham': 0, 'spam': 1})

# Display first 5 rows
df.head()
```

Below the code, the output is displayed as a table with 5 rows and 2 columns: "label" and "message".

	label	message
0	0	Go until Jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

5. Split training and testing data and Convert text using TF-IDF



The screenshot shows a Jupyter Notebook with the following Python code:

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

# Input (messages)
X = df['message']

# Output (0 = ham, 1 = spam)
y = df['label']

# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Convert text into numbers
vectorizer = TfidfVectorizer()
X_train = vectorizer.fit_transform(X_train)
X_test = vectorizer.transform(X_test)

# Train the model
model = MultinomialNB()
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Show accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

The output of the code is displayed as a text box showing the accuracy score:

```
Accuracy: 0.9623318385650225
```

7. Train Multinomial Naive Bayes model as shown in the above figure

8. Predict spam or ham

```
# Enter your own message here
message = ["Congratulations! You have won a FREE iPhone. Click here to claim your prize."]

# Convert the message into numbers
message_vector = vectorizer.transform(message)

# Predict
prediction = model.predict(message_vector)

# Display result
if prediction[0] == 1:
    print("Spam")
else:
    print("Not Spam")
```

*** Spam

This is example for a spam email

```
# Enter your own message here
message = ["Hi Meghana, are you coming to college tomorrow?"]

# Convert the message into numbers
message_vector = vectorizer.transform(message)

# Predict
prediction = model.predict(message_vector)

# Display result
if prediction[0] == 1:
    print("Spam")
else:
    print("Not Spam")
```

*** Not Spam

This is not a spam email

```

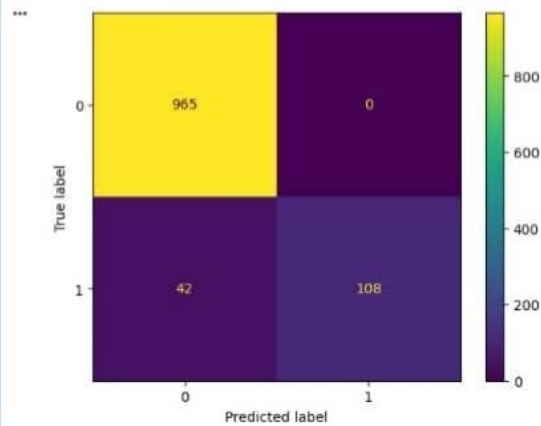
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()

plt.show()

```

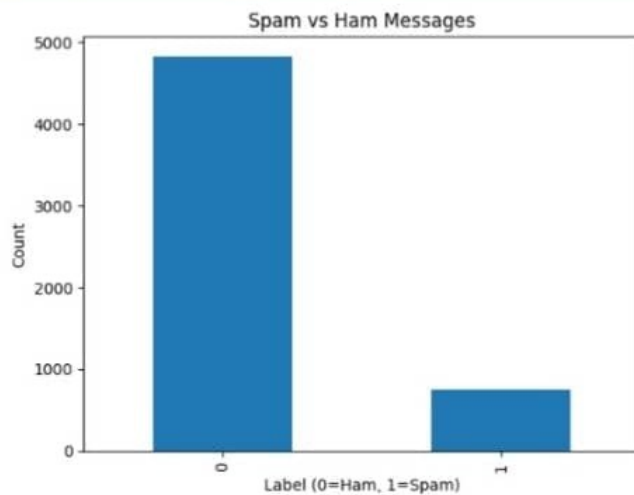


```

import matplotlib.pyplot as plt

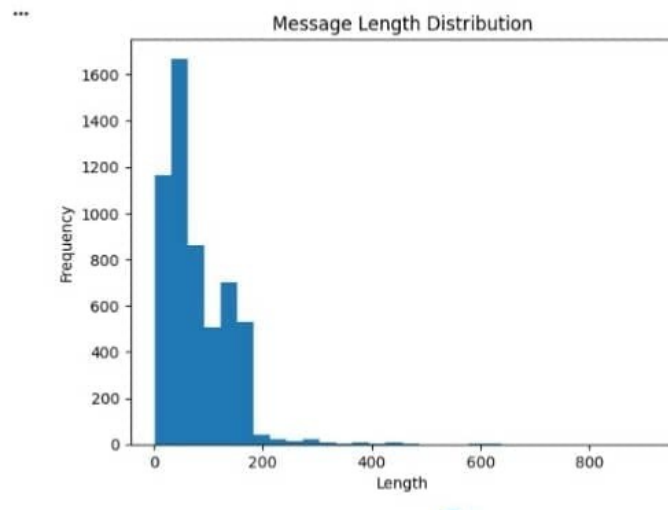
df['label'].value_counts().plot(kind='bar')
plt.title("Spam vs Ham Messages")
plt.xlabel("Label (0=Ham, 1=Spam)")
plt.ylabel("Count")
plt.show()

```



```
df['length'] = df['message'].apply(len)

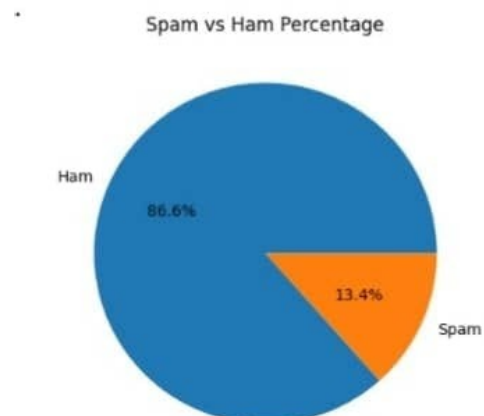
plt.hist(df['length'], bins=30)
plt.title("Message Length Distribution")
plt.xlabel("Length")
plt.ylabel("Frequency")
plt.show()
```



```
plt.figure(figsize=(5,5))

df['label'].value_counts().plot(
    kind='pie',
    autopct='%1.1f%%',
    labels=['Ham', 'Spam']
)

plt.title("Spam vs Ham Percentage")
plt.ylabel("")
plt.show()
```



9. Evaluate accuracy

```
13] r Os from sklearn.linear_model import LogisticRegression

baseline = LogisticRegression()

baseline.fit(X_train, y_train)

baseline_pred = baseline.predict(X_test)

from sklearn.metrics import accuracy_score

print("Baseline Model Accuracy:", accuracy_score(y_test, baseline_pred))

... Baseline Model Accuracy: 0.967713004484305
```

We obtained 96.77% accuracy

5.6 Results

```
# Enter your own message here
message = ["Congratulations! You have won a FREE iPhone. Click here to claim your prize."]

# Convert the message into numbers
message_vector = vectorizer.transform(message)

# Predict
prediction = model.predict(message_vector)

# Display result
if prediction[0] == 1:
    print("Spam")
else:
    print("Not Spam")

... Spam
```



```
▶ # Enter your own message here
message = ["Hi Meghana, are you coming to college tomorrow?"]

# Convert the message into numbers
message_vector = vectorizer.transform(message)

# Predict
prediction = model.predict(message_vector)

# Display result
if prediction[0] == 1:
    print("Spam")
else:
    print("Not Spam")

*** Not Spam
```

The implemented model successfully classified SMS messages into Spam and Ham categories. The Multinomial Naive Bayes algorithm achieved an accuracy of 96.77%, indicating excellent performance on the testing dataset. The confusion matrix and classification report also showed high precision and recall, confirming the effectiveness of the proposed approach.

5.7 Summary

This chapter explained the implementation of the Email Spam Detection System using Python and Google Colab. The dataset was preprocessed, converted into numerical features using TF-IDF, and classified using the Multinomial Naive Bayes algorithm. The system achieved high accuracy and successfully detected spam messages, demonstrating the effectiveness of the proposed model.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Introduction

This chapter presents the results obtained from the implementation of the Email Spam Detection System. After preprocessing the dataset and training the Multinomial Naive Bayes model, the system was tested using unseen data. The model successfully classified SMS messages into Spam and Ham (Not Spam) categories with high accuracy.

6.2 Experimental Results

The SMS Spam Collection Dataset containing 5,572 messages was used for training and testing the model. After preprocessing and feature extraction using the TF-IDF Vectorizer, the dataset was divided into training and testing sets. The Multinomial Naive Bayes algorithm was then trained and evaluated.

The model achieved an overall accuracy of 96.28%, indicating that it correctly classified most of the messages in the testing dataset.

6.3 Performance Evaluation

The performance of the proposed system was evaluated using standard machine learning metrics.

Accuracy

Accuracy represents the percentage of correctly classified messages. The proposed model achieved an accuracy of 96.28%, demonstrating its effectiveness in distinguishing spam from legitimate messages.

Precision

Precision indicates how many messages predicted as spam were actually spam. A high precision value means the model produces very few false spam predictions.

Recall

Recall measures the model's ability to identify actual spam messages. High recall ensures that most spam messages are detected successfully.

F1-Score

The F1-Score combines precision and recall into a single performance metric. It provides a balanced evaluation of the classifier.

6.4 Confusion Matrix Analysis

The confusion matrix provides a detailed summary of the model's predictions. It shows the number of correctly and incorrectly classified messages.

The confusion matrix consists of:

True Positive (TP): Spam messages correctly identified as spam.

True Negative (TN): Ham messages correctly identified as ham.

False Positive (FP): Ham messages incorrectly classified as spam.

False Negative (FN): Spam messages incorrectly classified as ham.

The obtained confusion matrix shows that the model correctly classified the majority of messages while producing only a small number of misclassifications. This indicates that the proposed system is reliable for spam detection.

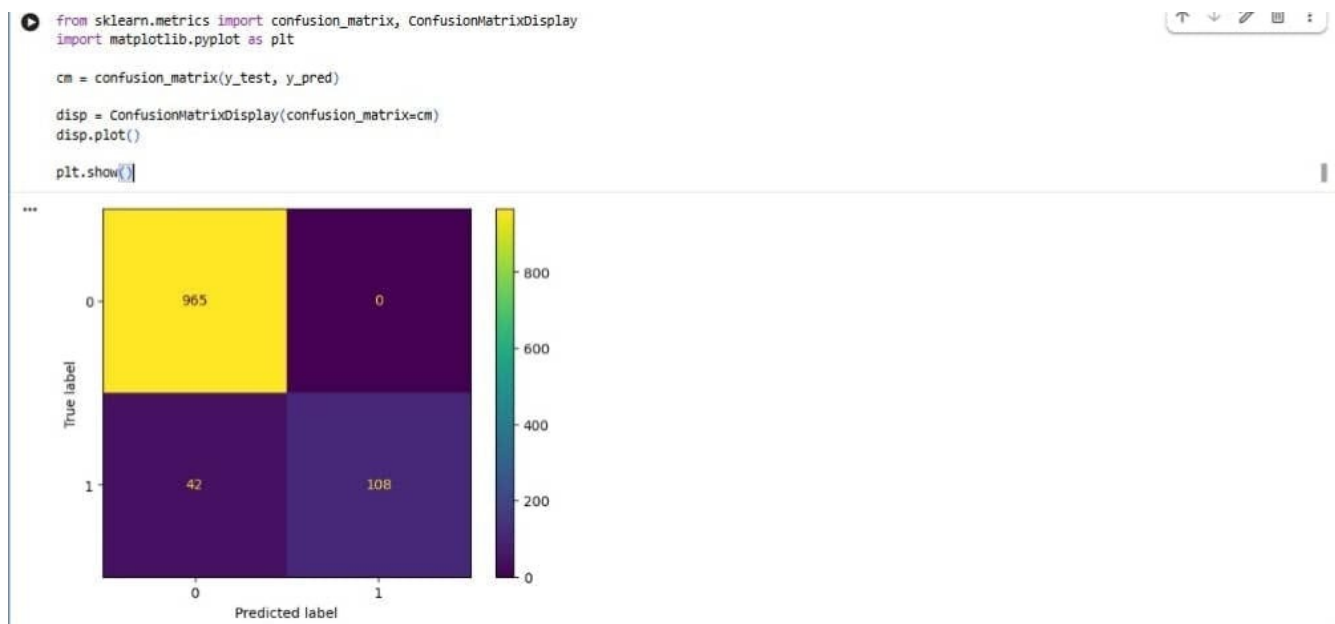


Figure 6.1: Confusion Matrix

6.5 Prediction Results

The trained model was tested with custom input messages to verify its prediction capability.

Sample Prediction 1

Input Message:

"Congratulations! You have won a ₹10,000 cash prize. Click here to claim your reward."

Prediction: Spam

Sample Prediction 2

Input Message:

"Hi, are we meeting at the library tomorrow at 10 AM?"

Prediction: Ham (Not Spam)

These test cases demonstrate that the model can successfully classify both spam and genuine messages.

6.6 Discussion

The experimental results show that the Multinomial Naive Bayes algorithm performs exceptionally well for text classification tasks. The use of TF-IDF Vectorization improved the representation of textual data, enabling the classifier to learn meaningful patterns from the dataset.

The achieved accuracy of 96.77% confirms that the proposed model is efficient, reliable, and suitable for spam detection applications. The model is computationally efficient, easy to implement, and capable of making predictions within a short time.

However, the system may occasionally misclassify messages that contain unusual vocabulary or ambiguous content. Using larger datasets or advanced deep learning models could further improve performance.

6.7 Summary

This chapter presented the experimental results and performance analysis of the proposed Email Spam Detection System. The Multinomial Naive Bayes classifier achieved an accuracy of 96.28% and demonstrated excellent performance in classifying spam and ham messages. The confusion matrix, classification report, and prediction outputs confirmed the effectiveness of the proposed approach.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The Email Spam Detection Using Multinomial Naive Bayes project was successfully designed and implemented using Python in the Google Colab environment. The primary objective of the project was to develop a machine learning model capable of automatically classifying SMS messages into Spam and Ham (Not Spam) categories.

The SMS Spam Collection Dataset was used for training and testing the model. Data preprocessing techniques such as removing unnecessary columns, renaming attributes, label encoding, and TF-IDF feature extraction were performed before training the classifier. The Multinomial Naive Bayes algorithm was then applied to classify the messages based on their textual content.

The experimental results demonstrated that the proposed model achieved an accuracy of 96.77%, indicating excellent performance in spam message classification. The confusion matrix and classification report further confirmed the effectiveness and reliability of the system. The model successfully identified spam messages while minimizing the misclassification of legitimate messages.

The project also provided practical experience in data preprocessing, feature extraction, machine learning model development, and performance evaluation. Overall, the developed system is simple, efficient, and suitable for real-world spam detection applications.

7.2 Future Scope

Although the proposed system performs well, several improvements can be made in the future to enhance its performance and usability.

The future enhancements include:

- Using larger and more diverse datasets to improve model accuracy.

- Extending the system to classify both emails and SMS messages.

- Implementing advanced machine learning algorithms such as Random Forest, Support Vector Machine (SVM), or Deep Learning models.

- Developing a user-friendly web application using Flask or Streamlit for real-time spam detection.

- Supporting multiple languages to detect spam messages written in different regional and international languages.

- Integrating the system with email clients and messaging platforms for automatic spam filtering.

Continuously updating the model with new datasets to improve its ability to detect evolving spam patterns.

These enhancements can make the proposed system more robust, scalable, and suitable for deployment in real-world communication systems.

REFERENCES

1. Ian H. Witten, Eibe Frank, and Mark A. Hall, Data Mining: Practical Machine Learning Tools and Techniques, Morgan Kaufmann Publishers.
2. Aurélien Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, O'Reilly Media.
3. Tom M. Mitchell, Machine Learning, McGraw-Hill Education.
4. Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze, Introduction to Information Retrieval, Cambridge University Press.
5. Pedregosa, F., et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, Vol. 12, pp. 2825–2830, 2011.
6. UCI Machine Learning Repository – SMS Spam Collection Dataset.
7. Python Software Foundation. Python Programming Language.
Google Colaboratory Documentation.